Problem Set C — Numerical Methods (Week + Integration)

Programming and Numerical Methods for Economics (ECNM)
The University of Edinburgh · School of Economics

Instructions. This problem set covers material from Week and integrates ideas from ear weeks: root-finding with `scipy.optimize`, unconstrained and constrained optimisation, and e applications (Solow model, consumer choice). You will need `numpy`, `scipy`, and `matplotl`

In [ ]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy import optimize
```

Question — Root-finding: supply and demand

Consider a market with inverse demand $P^D(Q) = - Q$ and inverse supply $ + Q$.

(a) Solve for the equilibrium analytically: set $P^D = P^S$ and find $Q^ $ and $P^ $ by hanc the answer as a comment).

(b) Define an excess-demand function `excess_demand(Q)` that returns $P^D(Q) - P^S(Q)$. equilibrium is the root of this function.

(c) Plot `excess_demand(Q)` for $Q \in [,]$. Verify visually that it crosses zero near analytical answer.

(d) Use `optimize.fsolve` to find the root numerically. Compare to your analytical answer.

In [ ]:
```
# Your answer here
```

Question — Root-finding: the Solow steady stat

In the Solow model, the steady-state capital per worker $k^ $ solves:

$$s \cdot A \cdot (k^)^\alpha - \delta \cdot k^ = $$

Use $A = $, $\alpha = . $, $\delta = . $, $s = . $.

(a) Define a function `solow_ss(k, s, A, alpha, delta)` that returns $s \cdot A \cdot k^
\delta \cdot k$.

(b) Plot the function for $k \in [. ,]$. Where does it cross zero?

(c) Use `optimize.fsolve` to find $k^ $. Compute the corresponding steady-state output $
A (k^)^\alpha$ and consumption $c^ = (-s) y^ $.

(d) Comparative statics: Repeat for $s = . , . , . , . $.
results in a dictionary and print a table showing s, $k^ $, $y^ $, and $c^ $. Which saving
gives the highest consumption?

In [ ]:
```
# Your answer here
```

Question — Unconstrained optimisation: compa
algorithms

Consider the **Rosenbrock function** in two dimensions: $$f(x_ , x_) = (- x_)^ +
(x_ - x_ ^)^ $$

The global minimum is at $(x_ , x_) = (,)$ with $f = $.

(a) Define `rosenbrock(X)` where `X` is a -element array.

(b) Starting from $x_ = (- ,)$, minimise using:

. BFGS (default `optimize.minimize`)
. Nelder–Mead

For each, print the solution, the function value at the solution, the number of function evaluations
(`res.nfev`), and whether the algorithm reported success.

(c) Now use the steeper variant $f(x_ , x_) = (- x_)^ + (x_ -
x_ ^)^ $ (coefficient changed from to). Does either method strug
$x_ = (- ,)$? Report function evaluations.

In [ ]:
```
# Your answer here
```

Question — Bounded optimisation: portfolio
allocation

An investor allocates wealth between two assets. Asset returns are:

• Asset : expected return $\mu_ = . $, variance $\sigma_ ^ =

- Asset : expected return $\mu_ = . $, variance $\sigma_ ^ =
- Covariance: $\sigma_{ } = . $

The investor chooses a weight $w \in [,]$ in Asset (with $ -w$ in Asset) t **minimise portfolio variance**:

$$\sigma_p^ (w) = w^ \sigma_ ^ + (-w)^ \sigma_ ^ + w(-w) \sigma_{ }$$

(a) Define `portfolio_var(w)` implementing the formula.

(b) Plot portfolio variance for $w \in [,]$. Where is the minimum approximately?

(c) Use `optimize.minimize` with `method='L-BFGS-B'` and bounds $w \in [,]$ to minimum-variance weight $w^ $. What is the portfolio's expected return at this weight?

(d) Now add a constraint: the portfolio must achieve an expected return of **at least %**. That \mu_ + (-w) \mu_ \geq . $. Use `method='SLSQP'` with the constraint. W the new optimal w?

In [ ]:
```
# Your answer here
```

Question — Root-finding meets simulation: the I curve

Consider a simple IS-curve model where output Y satisfies:

$$Y = C(Y) + I(r) + G$$

with consumption $C(Y) = c_ + c_ (Y - T)$, investment $I(r) = \bar{I} - d \cdot r$, gover spending G, and taxes T.

Parameters: $c_ = $, $c_ = . $, $\bar{I} = $, $d = = $, $T = $.

(a) Define `is_equation(Y, r)` returning $Y - C(Y) - I(r) - G$ (which equals zero in equilibriu

(b) For a given interest rate $r = . $, use `fsolve` to find equilibrium $Y^ $.

(c) Compute $Y^ $ for $r \in \{ . , . , \ldots, . \}$. Plot the IS curve ($ y-axis, Y on the x-axis). Does it slope downward as theory predicts?

(d) Fiscal multiplier: Increase G from to (keeping $r = . What is the new $Y^ $? Compute $\Delta Y / \Delta G$. Compare to the theoretical Keynesian multiplier $ /(- c_)$.

In [ ]:
```
# Your answer here
```

Question — Putting it all together: the N-dimens Rosenbrock

The Rosenbrock function generalises to N dimensions:

$$f(\mathbf{x}) = \sum_{i= }^{N- } \left[(- x_i)^ + (x_{i+ } - x_i^)^
\right]$$

In the Week lecture, this was written out term by term for $N= $. Your job is to make it

(a) Write a function `rosenbrock_nd(X)` that works for **any** length of input array `X`, using a lo vectorised NumPy).

(b) Verify that for $N= $ your function matches the D Rosenbrock from Question a test points: $(,)$, $(,)$, $(- ,)$.

(c) Minimise the $N= $ Rosenbrock using BFGS from $x_ = \mathbf{ }$. Report the function value, and number of evaluations. Is the solution close to $(, , \ldots,)$?

(d) Time BFGS vs Nelder–Mead for $N= $ (use `%%time` or `import time`). Which is fas Which uses fewer function evaluations?

In [ ]:
```
# Your answer here
```

End of Problem Set C.